

MUSUBI PLATFORM

Musubi プラットフォーム 方式設計書（基本設計）

システム方式・技術アーキテクチャ — 実装の実態に基づく設計記述

版数 **v0.1（ドラフト）**

作成日 **2026-06-29**

対象 **管理/ポータル/公開サイト/連携**

関連 **要件定義書 v0.3**

本書は「どう実現するか（方式）」を示す。要件定義書(WHAT)に対し本書はHOW。未実装/予定は明記する。

本書の目的と位置づけ

- 目的：Musubi 事業システムの技術方式（アーキテクチャ・構成・処理方式・連携・運用）を記述し、開発・保守・引き継ぎの基準とする。
- 位置づけ：要件定義書(WHAT)の実現方式(HOW)。要件で挙げた未確定/追加論点に対応する方式上の現状と課題も示す。
- 対象：管理アプリ(musubi-admin)・顧客ポータル(musubi-portal)・公開サイト群・定期処理・外部SaaS連携。
- 凡例：本文中の「予定」「課題」は未実装または要改善。確定済みは平文で記す。コード/テーブル名は実装に対応。

ARCHITECTURE

システム方式の全体像

管理アプリ musubi-admin

Next.js 15 / React 19 / TS
Vercel (SSR・Server Actions)
営業/制作/予約/口コミ/課金/レポート

顧客ポータル musubi-portal

Next.js 15 / Vercel
店主：口コミ承認・レポート閲覧
Supabase Auth(owner)

公開サイト（店舗別）

静的HTML / Cloudflare Pages
<店舗>.musubiweb.com・SSL
ネット予約・GA4計測

データ・認証基盤 Supabase

Postgres(Tokyo) + RLS / Auth(staff・owner)
Storage(site-assets)
leads/stores/reservations/reviews/reports/contracts

処理・連携

Cron(月次レポート/コスト/口コミ下書き)
Stripe・Resend・GA4・GBP・Cloudflare・Anthropic

方式の要：UIとデータはSupabaseに集約、公開サイトのみ静的分離。重い処理はサーバ(Server Actions/Route/Cron)に寄せる。

STACK

ソフトウェア方式（技術スタック）

区分	採用技術・方式
フロント/サーバ	Next.js 15(App Router)・React 19・TypeScript。Server Actions + Route Handlers。@supabase/ssr
スタイル	共通globals.css の独自クラス／CSS Module（Tailwindは一部のみ・原則 独自CSS方式）
データ	Supabase Postgres(Tokyo)・行レベルセキュリティ(RLS)・Storage・Auth
公開サイト	プレースホルダ式テンプレート→TSレンダリングエンジンで静的HTML生成→Cloudflare Pages
AI	Anthropic Messages API（tool-use 構造化出力）。sonnet-4-6(制作)／haiku(口コミ)
連携	Stripe(課金)・Resend(メール)・GA4 Data API(集計)・Google Business Profile(口コミ)
基盤	Vercel(管理/ポータル)・Cloudflare(Pages/DNS)・GCP(WIF for GA4)

アプリ構成（管理アプリ）

- 画面：dashboard / crm(営業リスト) / stores(顧客管理) / builder(サイト制作) / reservations(ネット予約) / reviews(口コミ返信AI) / billing(請求) / costs / report / settings。
- レイアウト：AdminShell（左ナビ＋トップバー）。共通UIは globals.css。
- データアクセス：サーバ側で @supabase/ssr のユーザクライアント（RLS適用）。サービスロールは webhook・公開API・Storage 等の限定箇所のみ。
- 更新：Server Actions（フォーム送信）＋ revalidatePath。送信中UIは useFormStatus。

ポータル / 公開サイト

顧客ポータル

reviews(承認)・report(閲覧)。owner権限・自店のみ。

公開サイト

index/reserve の2ページ静的HTML。予約JSはAPI駆動、未接続時はデモ動作にフォールバック。

データ方式（主要テーブル）

テーブル	役割・主な属性／方式
leads	営業リード。スコア/ランク自動算出。staff限定RLS
stores	店舗。template・content(jsonb)・site_project・site_domain・公開状態
reservations	予約。date/time/party/seat/course・status・res_no。在庫=定員－確定予約
reviews	口コミと返信下書き。status(pending/drafted/approved/posted)・flagged
reports	月次指標。アクセス/電話/予約。GA4集計をupsert・自動公開
contracts / invoices	契約・請求。Stripe price/period/支払状態を同期
staff_members	運営スタッフ。認可(requireStaff)の基点

方式メモ：サイト内容は stores.content(jsonb) に集約しテンプレへ流し込む。画像は Storage(site-assets) の公開URLを content に保持。

PROCESSING

処理方式（同期・非同期）

同期：Server Actions

管理/ポータルフォーム送信
保存・公開・取消・承認 等
RLS or 認可ラップで実行

同期：Route Handlers

/builder/upload・ai-fill
/api/reserve/availability・book
CORS・認証は経路ごと

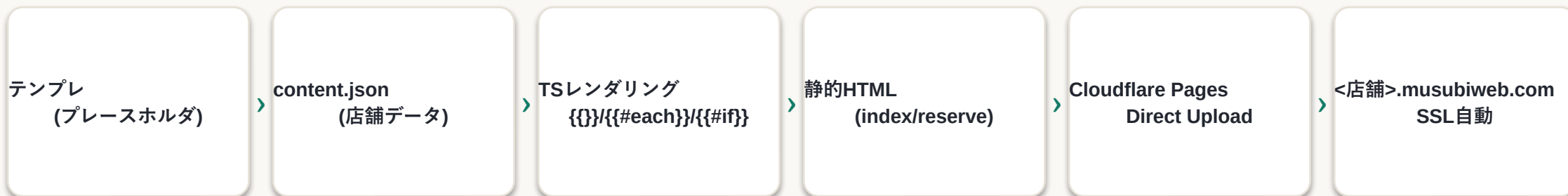
非同期：Cron

月次レポート集計(0 3 1 * *)
コストアラート(日次)
ロコミ下書き(予定:GBP連動)

認証境界(middleware)

- middleware が全リクエストで getUser()。未認証は /login へリダイレクト。
- 例外：/api/cron/*（CRON_SECRET の Bearer 照合）と /api/reserve/*（公開・CORS）は認証を素通し。
- Cron は Vercel スケジューラが Authorization: Bearer CRON_SECRET を付与して起動。

サイト生成・公開方式



- レンダリング：Pythonの旧ビルダを TypeScript に移植（isomorphic）。ブラウザ＝即時プレビュー、サーバ＝公開生成で共用。
- 公開：wrangler 準拠の Direct Upload（ファイルハッシュ=blake3 → upload-token → check-missing → upload → upsert-hashes → deployment）。
- ドメイン：Pages の独自ドメイン割当 + Cloudflareゾーンに CNAME(proxied) を自動作成。削除時は Pages + CNAME を削除。

ネット予約方式（在庫型）

- 空き算出：時間帯ごとに「定員(平日/週末・繁忙補正) - 確定予約人数」。月=日別ステータス、日=スロット別残席を返す。
- 予約確定：book_reservation RPC。スロット単位の pg_advisory_xact_lock で容量を原子的にチェックし挿入（オーバーストッキング防止）。
- 公開API：/api/reserve/{availability,book}。CORS開放・認証なし・store は site_project で検証。確定後 Resend で来店客/店舗へ通知。
- 公開サイト側：予約JSが api_base/store を注入されていれば実APIを使用。未注入(プレビュー)はローカルのデモ動作にフォールバック。
- 課題（要件 ADDENDUM）：レート制限/CAPTCHA/冪等キー・メール検証・状態機械(仮→確定→来店/取消/No-show)は今後の方式拡張。

AI方式（Claude）

用途	方式
口コミ返信下書き	Messages API + システムプロンプト。口コミを信頼境界外として <review> で隔離(インジェクション対策)。低評価はflagged
サイト下書き生成	参考URLを取得→HTML除去 + 画像抽出→tool強制で content を構造化出力。画像は候補URLのみ採用(捏造防止)
安全方式	SSRF対策(DNS解決で私的IP拒否 + リダイレクト不追従)・出力エスケープ・人手(店主)承認を前提
モデル	制作=sonnet-4-6／口コミ=haiku（環境変数で切替）。失敗時はテンプレ/人手確認にフォールバック

方式メモ：顧客PII(予約のtel/email等)はAIへ送らない。AI生成物は公開前に人間承認する運用を方式上の前提とする。

月次レポート方式 (GA4)

- 集計：GA4 Data API (@google-analytics/data)。1プロパティに集約し hostName で店舗識別（顧客には加工レポートのみ提示）。
- 認証：鍵レスの Workload Identity Federation。VERCEL_OIDC_TOKEN → GCP STS → サービスアカウントになりすまし。
- 実行：月次 Cron(0 3 1 * *) が前月分を runReport→reports に upsert。未公開分のみ published_at を自動セットし一度だけ通知(幂等)。
- 計測：公開サイトに GA4 タグ標準同梱（電話タップ・予約送信イベント）。measurement_id 空なら非出力。
- 課題：時系列スナップショット(MRR/前年比)・指標定義・bot除外・TZ境界の確定（要件 ADDENDUM 参照）。

認証・認可方式

層	方式
プラットフォーム	Vercel Deployment Protection（管理/ポータルの前段ゲート）
認証	Supabase Auth（メール/パスワード）。middleware が getUser() で検証し未認証は /login
認可(運営)	staff_members 照合の requireStaff()。課金等の操作を staff 限定
認可(店主)	owner_user_id と対象 store の一致を検証（口コミ承認・予約取消の IDOR ガード）
分離	RLS で store スcope。read=RLS下ユーザクライアント、write=本人確認後サービスクライアント
公開API	認証なし・CORS開放だが store を site_project で検証（書込は service role）

課題：RLS を全テーブルでマイグレーション管理し、テナント分離の自動テストを整備（現状 reservations 以外はDashboard設定依存）。

外部連携方式

連携先	方式
Stripe	契約=サブスク。Webhook(署名検証・stripe_invoice_id で冪等)で入金/失敗/解約をDB同期。失敗時はResend案内
Resend	予約確認/通知・レポート公開通知・課金案内。送信元 send.musubiweb.com（SPF/DKIM設定済）
GA4	Data API で月次集計（上記レポート方式）
Google Business Profile	口コミ取得・承認後の投稿（API 利用承認待ち＝現状手動代替）
Cloudflare	Pages Direct Upload・独自ドメイン割当・DNS(CNAME)操作を API で実行
Anthropic	上記 AI 方式

ネットワーク・ドメイン方式

- ドメイン：musubiweb.com の DNS を Cloudflare で管理（NS=emma/miguel.ns.cloudflare.com）。Squarespace から移管済。
- サブドメイン：各店 <店舗>.musubiweb.com を Cloudflare Pages の独自ドメインとして割当。CNAME → <project>.pages.dev(proxied)。SSL 自動。
- 既存レコード：主サイト(apex/www)・Google Workspace メール(MX/SPF/DKIM/DMARC)は Cloudflare 上で保持（移管時に複製）。
- メール送信：send.musubiweb.com を Resend 用に SPF/DKIM 設定。DMARC 方針の明文化は課題。

OPERATION

運用・監視・バックアップ方式

CI/CD

GitHub push → Vercel 自動デプロイ
環境変数は Vercel 管理
構築ログ・ロードマップを随時更新

監視

UptimeRobot 3点(管理/ポータル/公開)
コスト暴走アラート(80/100%・日次)
障害ランブック整備

バックアップ

論理BU(全テーブルJSON)復元手順
Supabase標準BU
PITR/暗号化保管は課題

- デプロイ方式の留意：自動デプロイは壊れたコードも本番化しうる。ロールバック(Instant Rollback)・公開前確認・スモークテストは方式上の課題。
- Cron 監視：成否の死亡通知型監視・幕等・突合(Stripe ↔ DB)は今後の方式強化（要件 ADDENDUM）。

セキュリティ方式（実装済み）

観点	方式
出力(XSS)	メール/HTML出力でユーザ入力をエスケープ
アップロード	画像は拡張子 allowlist から Content-Type 決定（client MIME不採用・SVG除外）
SSRF	AI取得URLを DNS解決して私的/予約IPを拒否＋リダイレクト不追従
IDOR	予約取消・口コミ承認で店舗スコープ＋本人確認
インジェクション	口コミ等の外部テキストを信頼境界外として隔離
境界/CORS	管理/ポータルは認証必須。公開APIは CORS 限定＋store検証

方式上の残課題：予約APIの多層防御・全RLSのコード化と分離テスト・監査ログ・PII暗号化/保持・シークレット運用（要件 ADDENDUM）。

性能・可用性・拡張の方式

- 性能：公開サイトは静的配信(Cloudflare CDN)で高速。プレビューはクライアント生成で即時。重い集計は Cron に分離。
- 可用性：マネージドSaaS(Vercel/Cloudflare/Supabase)に依拠。公開サイトはデータ層と分離し、管理系障害が公開を止めない構成。
- 拡張：サーバレスで水平拡張。テンプレート追加・モジュール追加で機能拡張。content(jsonb)で店舗ごと差分を吸収。
- 課題：コンポーネント別SLO・RTO/RPO・PITR・TZ統一・在庫モデル拡張・content版管理（要件 ADDENDUM で具体化）。

開発・テスト・リリース方式

- リポジトリ：musubi-admin / musubi-portal（アプリ・private）、musubi-demos（ハブ・public）、local-web-sales（ドキュメント）。公開サイトは Cloudflare Pages プロジェクト。
- ブランチ/デプロイ：master へ push → Vercel 本番自動デプロイ。アプリ反映はユーザ指示時に push、ドキュメントは随時 push。
- 品質：実装時に tsc(型) ・ build を必ず通す。UI は実描画(ヘッドレス)で確認。数値は捏造せず未接続は「—」。
- 検証：DBマイグレーションは Management API で本番適用。重要機能はキー投入後に本番でE2E実証。
- 課題：自動テスト(RLS分離・予約整合・回帰)・ステージング・公開前承認は今後整備。

方式上の技術的課題（優先）

先に固める（設計と不可分）

- 予約APIの多層防御（レート制限/CAPTCHA/幂等）
- 全RLSのコード化+テナント分離テスト
- 課金⇄公開/予約の状態機械（自動連動）
- レポート時系列スナップショット
- タイムゾーン統一（UTC保存/JST境界）
- シークレット運用（restricted key・一括ignore）

中期：方式を成熟させる

- SLO/RTO・PITR・Cron/Stripe監視と突合
- 在庫モデル拡張・ホスト名エイリアス
- content の schema_version・バリデーション
- 監査ログ・PII暗号化/保持
- 公開サイトのSEO/構造化データ/CWV
- 自動テスト・ステージング・ロールバック

詳細は要件定義書 v0.3「検討漏れ・追加論点」と対応。本書はその方式面の現状と対応方針を示す。

方式設計の要点

- UIとデータは Supabase に集約、公開サイトのみ静的分離（障害波及を抑制）。
- 公開はテンプレ→content→TSレンダリング→Cloudflare Pages Direct Upload で内製・1操作化。
- 予約は在庫型(RPC+ロック)、AI/レポートはサーバ・Cronに分離、認証は middleware で集中制御。
- 確定済み方式の上に、予約API防御・RLSコード化・課金連動・TZ・スナップショット等を順次強化する。